

CS 6501 — Week 3 Cheat Sheet: MAPF and Scalability

Big Picture

Week 3 focuses on **Multi-Agent Path Finding (MAPF)** and the algorithmic challenges that arise when planning collision-free paths for many robots. The central theme is scalability: why naïve optimal planning fails, how Conflict-Based Search (CBS) improves performance, and how symmetry is the main remaining bottleneck.

Multi-Agent Path Finding (MAPF)

MAPF asks for collision-free paths for multiple agents from start to goal positions on a shared environment (often a grid), while minimizing a global cost such as makespan or sum of costs.

Key assumptions:

- Discrete space and time
- Agents occupy vertices and move along edges
- Collisions occur if agents share a vertex or edge at the same time

MAPF is NP-hard and becomes intractable under naïve formulations.

Joint-State A* (Baseline)

The most direct approach is to plan in the joint-state space:

$$(x_1, x_2, \dots, x_N)$$

Problems:

- State space grows exponentially with number of agents
- Even optimal A* fails beyond a small number of robots
- Computationally infeasible for realistic systems

Key insight: Optimality alone does not imply scalability.

Conflict-Based Search (CBS)

CBS improves scalability by separating planning and collision resolution.

High-level idea:

1. Plan paths for each agent independently
2. Detect collisions between agents
3. Resolve collisions by adding constraints
4. Replan only the constrained agent

Instead of searching over trajectories, CBS searches over **constraints**.

CBS Constraint Branching

When a collision occurs between agent i and j at location v and time t :

- Branch 1: forbid agent i from (v, t)
- Branch 2: forbid agent j from (v, t)

Each branch yields a new planning problem with minimal changes.

Why CBS Still Struggles

Although CBS scales better than joint-state search, it still suffers from:

- High branching factor
- Repeated exploration of equivalent solutions
- Poor performance in dense environments

The root cause is **symmetry**.

Collision Symmetries

A symmetry occurs when multiple agents can resolve a conflict in interchangeable ways that lead to equivalent outcomes.

Examples:

- Corridor swaps
- Rectangle crossings
- Target-area congestion

CBS treats symmetric resolutions as distinct, causing redundant search.

Key idea: Symmetry causes combinatorial explosion even in CBS.

Symmetry-Breaking Constraints

Rather than branching on individual agent constraints, symmetry-breaking constraints:

- Identify sets of equivalent collision resolutions
- Eliminate entire equivalence classes at once
- Preserve completeness and optimality

This dramatically reduces the CBS search tree.

Multi-Value Decision Diagrams (MDDs)

MDDs compactly represent all shortest paths for an agent:

- Layered by timestep
- Nodes represent reachable locations
- Edges represent valid transitions

By comparing MDDs across agents, symmetric interactions can be detected automatically.

Execution vs Planning

MAPF planning is discrete and offline, but real robots:

- Have continuous dynamics
- Experience delays and disturbances

Bridging strategies include:

- Reachability envelopes
- Temporal Plan Graphs (TPGs)
- Enforcing agent ordering only at critical vertices

Relation to the Course

Week 3 emphasizes:

- Algorithmic scalability
- Centralized planning with structured constraint reasoning
- Tradeoffs between optimality and practicality

This contrasts with:

- CBFs (continuous, reactive safety)
- MPC (trajectory optimization)
- MARL (learning-based coordination)

Key Takeaways

- Joint-state planning is optimal but unscalable
- CBS resolves collisions through constraint search
- Symmetry is the main scalability bottleneck
- Symmetry-breaking constraints significantly improve performance
- MAPF must interface carefully with real robot execution

One-Sentence Summary

MAPF scales by resolving collisions through constraint search rather than joint planning, and modern advances focus on eliminating symmetric redundancy to enable large multi-robot systems.